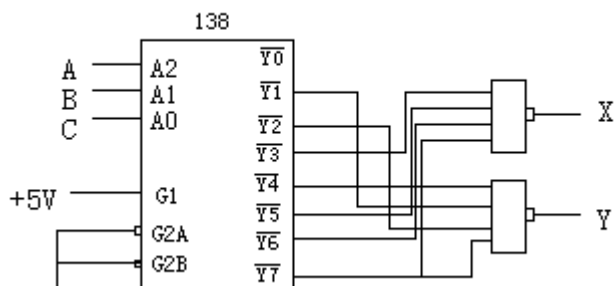


1、化简 $F = \overline{\overline{AB} + ABC + A(B + \overline{AB})}$ (5分)

答: $F = 0$

2、分析以下电路,说明电路功能。(10分)

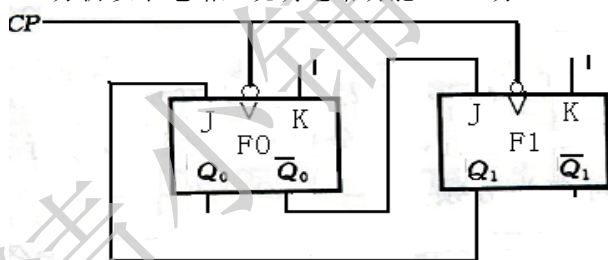


解: $X = \sum m(3,5,6,7)$ 2分
 $Y = \sum m(1,2,4,7)$

A	B	C	X	Y
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

该组合逻辑电路是全加器。以上8分

3、分析以下电路,说明电路功能。(10分)



解: (1)、 $J_1 = \overline{Q_0}$, $J_0 = Q_1$, $K_0 = K_1 = 1$ 3分

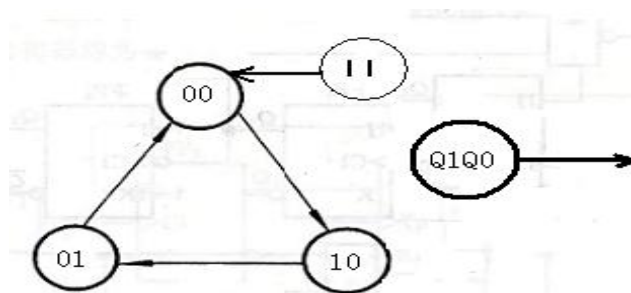
(2)、 $Q_{1n+1} = \overline{Q_0} \overline{Q_{1n}}$

$Q_{0n+1} = Q_1 \overline{Q_{0n}}$ 2分

(3)、 2分

$Q_{1n} Q_{0n}$	$Q_{1n+1} Q_{0n+1}$
0 0	1 0
0 1	0 0
1 0	0 1
1 1	0 0

(4)、



2分

该电路是3进制减法计数器 1分

4、设计一个带控制端的组合逻辑电路，控制端 $X=0$ 时，实现 $F = A \oplus B$ ，控制端 $X=1$ 时，实现 $F = \overline{AB}$ ，用与非门及反相器实现。(15分)

解：

(1)、真值表 (8分)

X	A	B	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

(2)、卡诺图、代数式：(4分)

$$F = \overline{AB} + A\overline{B} + X\overline{A}$$

$$\text{或 } F = \overline{AB} + A\overline{B} + X\overline{B}$$

(3)、画电路图：(3分) 略

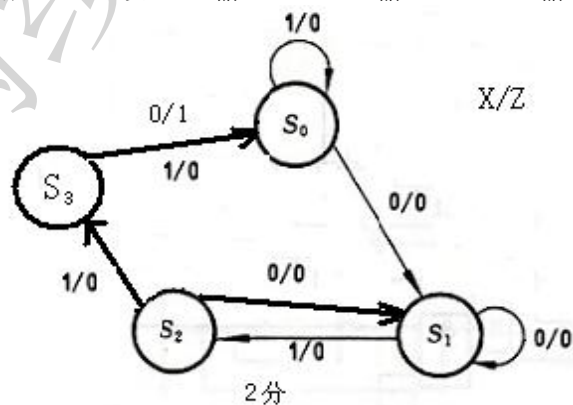
5、(1) 用 D 触发器设计一个 0110 序列检测器，X 为序列输入，Z 为检测输出，其关系如下。(15分)

X: 1011010110110

Z: 0000100001000

(2) 用 Verilog 语言描述实现以上功能的电路。

解：(1)、设 S0: 输入 1, S1: 输入 0, S2: 输入 01, S3: 输入 011, S4: 输入 0110



2分

S4 与 S0 等价，状态图如上。

(2)、列状态转换真值表及驱动表 每填对 1 格给 1 分 (8 分)

X	Q1n	Q0n	Q1n+1	Q0n+1	D1	D0	Z
0	0	0	0	1	0	1	0
0	0	1	0	1	0	1	0
0	1	0	0	1	0	1	0
0	1	1	0	0	0	0	1
1	0	0	0	0	0	0	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	0
1	1	1	0	0	0	0	0

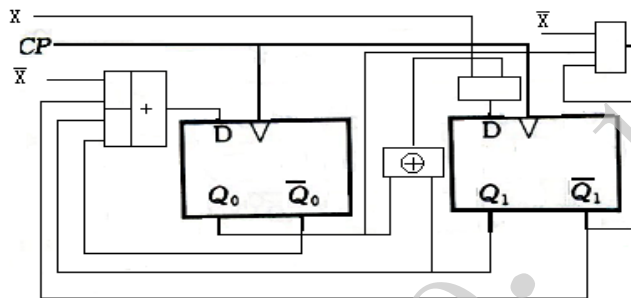
求驱动方程及输出方程，每个方程 1 分 (3 分)

$$D_1 = X(Q_1 \oplus Q_0)$$

$$D_0 = Q_1 \bar{Q}_0 + \bar{X} \cdot \bar{Q}_1$$

$$Z = \bar{X} Q_1 Q_0$$

电路图 2 分



参考 Verilog 程序:

```

module sequence_detector(
    input wire clk,
    input wire rst,
    input wire X,
    output reg Z
);
    // 定义状态
    reg [1:0] state;

    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

    // 状态转移逻辑
    always @(posedge clk or posedge rst) begin
        if (rst) begin

```

```

state <= S0;

Z <= 1'b0;

end

else begin

case (state)

S0: if (X == 1'b0) state <= S1; else state <= S0;

S1: if (X == 1'b1) state <= S2; else state <= S1;

S2: if (X == 1'b1) state <= S3; else state <= S1;

S3: state <= S0;

default: state <= S0;

endcase

end

end

// 输出逻辑

always @(*) begin

if (state == S3)

Z = 1'b1;

else

Z = 1'b0;

end

endmodule

```